भारतीय प्रौद्योगिकी संस्थान जम्मू
INDIAN INSTITUTE OF TECHNOLOGY JAMMU

INDIAN INSTITUTE OF TECHNOLOGY JAMMU

Department of Computer Science and Engineering

Course: Computer Vision — **Max Marks:** 100 — **Format:** Jupyter Notebook (.ipynb)

---

1 Objective

The objective of this assignment is to explore multi-scale image representations and construct a scale-invariant blob detector from fundamental principles. Given a grayscale image, you will build a scale-space using Gaussian filtering and downsampling, subsequently detecting visual blobs via scale-normalized Laplacian-of-Gaussian (LoG) responses.

Sample Input Image

For this assignment, use a grayscale image containing multiple blob-like structures at different spatial scales. The image should preferably contain objects of varying sizes, contrast levels, and spatial locations so that the behavior of the scale-space representation and blob detector can be clearly analyzed.

An example input image is shown in Fig. 1.



Figure 1: Example grayscale image containing blob-like structures at multiple spatial scales for scale-space and LoG-based blob detection.

2 Tasks & Implementation Instructions

Task 1 — Gaussian Scale Space (15 Marks)

Given an input grayscale image $I(x, y)$, generate Gaussian-smoothed images for at least five distinct scale values: $\sigma \in \{1, 2, 4, 8, 16\}$.

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y) \tag{1}$$

Display all filtered outputs. Discuss the visual impact of increasing σ on image fine details, high-frequency noise, edge boundaries, and structures across varying spatial scales.

Task 2 — Multi-Resolution Representation (10 Marks)

Construct a multi-resolution image pyramid by iteratively applying Gaussian smoothing followed by downsampling by a spatial factor of 2.

[leftmargin=*]

- Display the resulting images across all pyramid levels and state their spatial dimensions.
- Explain the mathematical and practical necessity of applying Gaussian anti-aliasing filters prior to downsampling.

Task 3 — Laplacian-of-Gaussian (LoG) Response (20 Marks)

For each scale level σ, compute the scale-normalized Laplacian-of-Gaussian response:

$$R(x, y, \sigma) = \sigma^2 \left| \nabla^2 L(x, y, \sigma) \right| \tag{2}$$

Display the computed response maps across all scales and discuss how scale normalization (σ^2) guarantees true scale invariance.

Task 4 — 3D Scale-Space Extrema Detection (20 Marks)

Formulate a 3D scale-space volume over (x, y, σ).

[leftmargin=*]

- Identify local extrema by comparing each candidate pixel with its 26-neighborhood (8 spatial neighbors at the same scale, 9 at the scale above, and 9 at the scale below).
- Apply a suitable threshold to filter out weak background noise and low-contrast extrema.

Task 5 — Blob Radius Estimation & Visualization (10 Marks)

For each verified local extremum (x, y, σ), estimate the characteristic spatial radius using:

$$r \approx \sqrt{2}\,\sigma \tag{3}$$

Visualize the detected features by overlaying circles of radius r centered at (x, y) on the original image.

Task 6 — Scale Invariance Analysis (15 Marks)

Evaluate the robust performance of your detector under scale transformations:

[leftmargin=*]

1. Create resized versions of the test image using scale factors $\mathbf{S} \in \{1.0, 0.75, 0.50, 0.25\}$.
2. Run your blob detector independently on each image variant.
3. Compare spatial detections across scales. Address whether the detector reliably identifies the same semantic structures across resizing operations and analyze any discrepancies.

Task 7 — Quantitative Evaluation & Parameter Discussion (10 Marks)

Construct a quantitative summary table formatted as follows:

Table 1: Quantitative Blob Detection Performance Across Resized Images

Image Variant	**Resize Factor**	**Detected Blobs**	**Avg. Radius ($\bar{r}$)**	**Detection Threshold**
Original	1.00	—	—	—
Variant 1	0.75	—	—	—
Variant 2	0.50	—	—	—
Variant 3	0.25	—	—	—

Provide a detailed discussion on how the number of scale octaves, Gaussian scale step σ, downsampling factor, detection threshold, and input resolution affect performance.

3 Implementation Constraints

[Strict Policy on Core Functions] You **must implement** the scale-space generation, Laplacian/DoG response, and 3D non-maximum suppression algorithm **from scratch**.

[leftmargin=*]

- **Prohibited Functions:** Do not use high-level feature detectors such as `cv2.SimpleBlobDetector`, `skimage.feature.blob_log`, SIFT, SURF, or ORB modules.

- **Permitted Libraries:** standard numeric and signal processing modules including `NumPy`, `SciPy` (e.g., convolution/filtering operations), `OpenCV` (basic image I/O and resizing), and `Matplotlib`.

4 Bonus Challenge (+10 Marks)

Implement a **Difference-of-Gaussians (DoG)** approximation to the LoG operator:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) \approx (k - 1)\sigma^2 \nabla^2 L(x, y, \sigma) \tag{4}$$

Compare the computational speed, feature location accuracy, and response quality between your LoG and DoG implementations.

5 Submission Guidelines

Submit a single well-documented Jupyter Notebook (`.ipynb`) containing:

[leftmargin=*]

1. Modular, well-commented Python code.
2. Visual outputs for all tasks (Scale-space, Pyramid, LoG maps, 3D extrema, final overlay).
3. The experimental result table (Task 7) and comprehensive analytical discussions.